



Natural Language Processing: Engineering Tricks

HSE Faculty of Computer Science
Machine Learning and Data-Intensive Systems

Murat Khazhgeriev



Table of Content

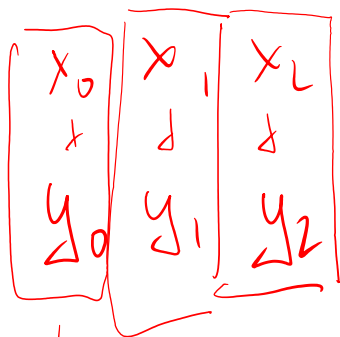
- GPU Architecture 101
- Engineering tricks for attention
- Intro to quantization
- Engineering tricks for decoding
- Miscellaneous



Table of Content

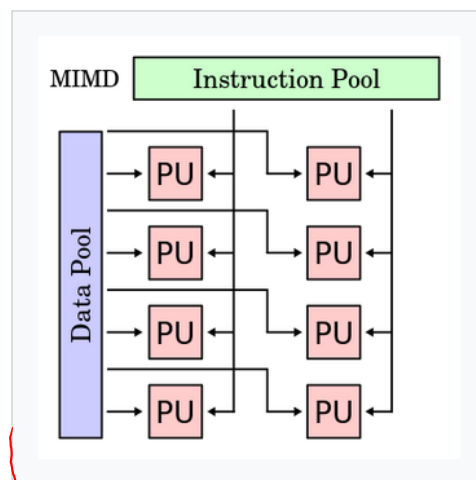
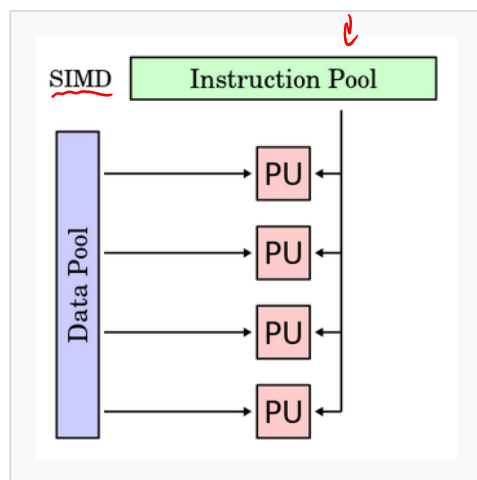
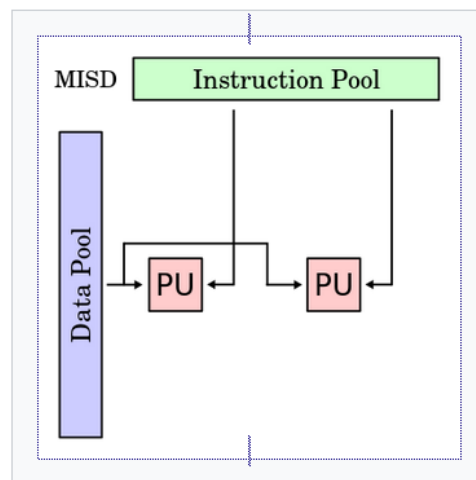
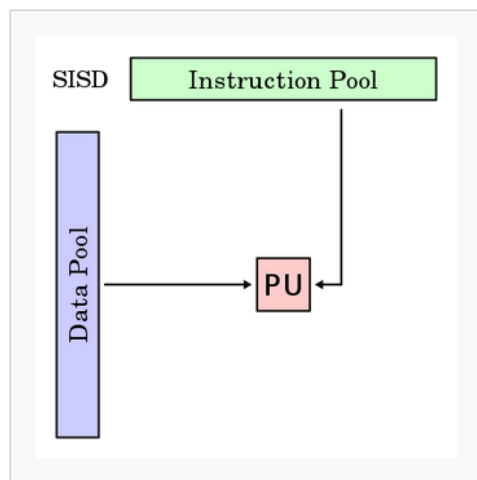
- **GPU Architecture 101**
- Engineering tricks for attention
- Intro to quantization
- Engineering tricks for decoding
- Miscellaneous

GPU Architecture Fundamentals

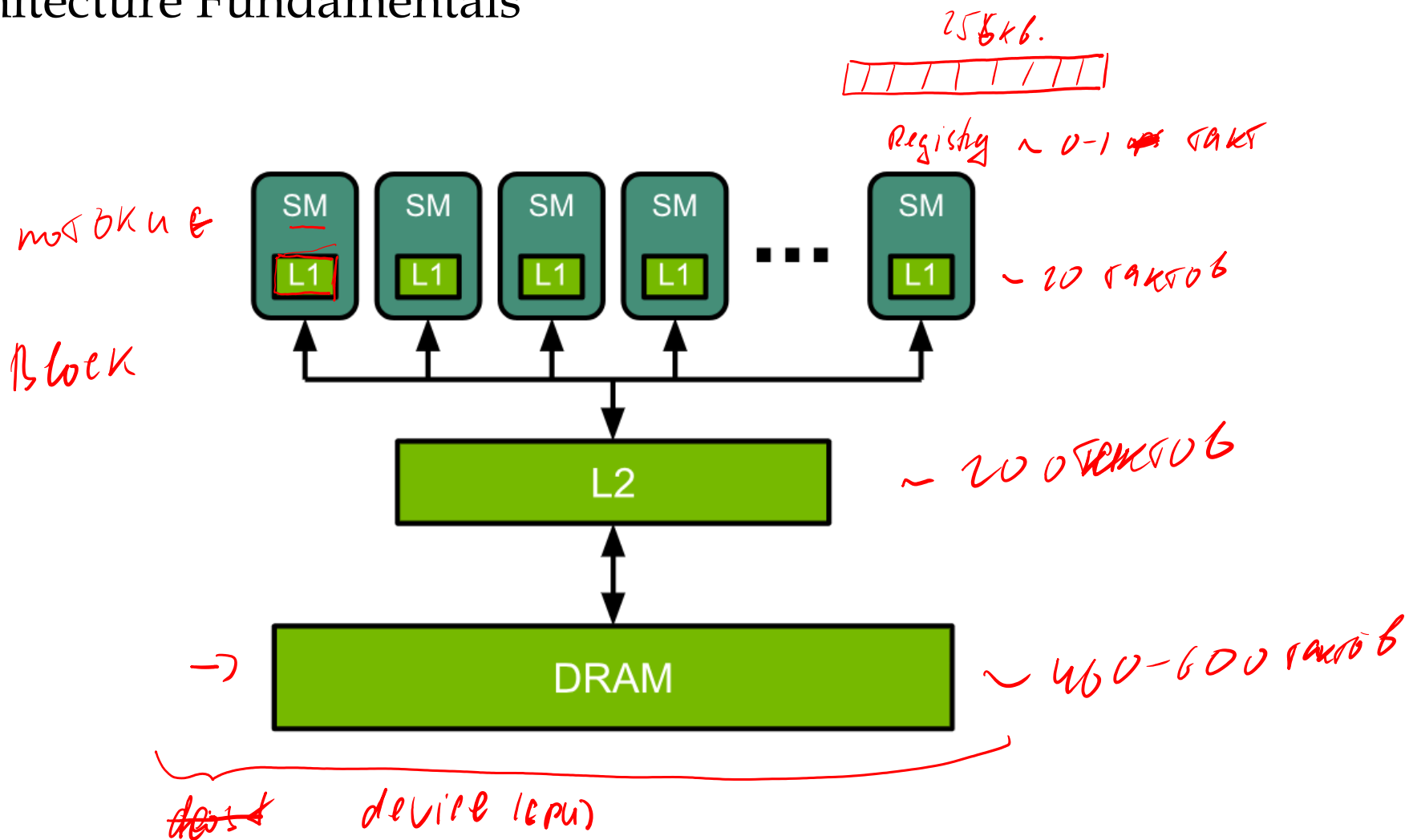


→ $x_0 \ x_1 \ x_2$
 x

→ $y_0 \ y_1 \ y_2$



GPU Architecture Fundamentals



GPU Architecture Fundamentals

work
in
thread

SIMT

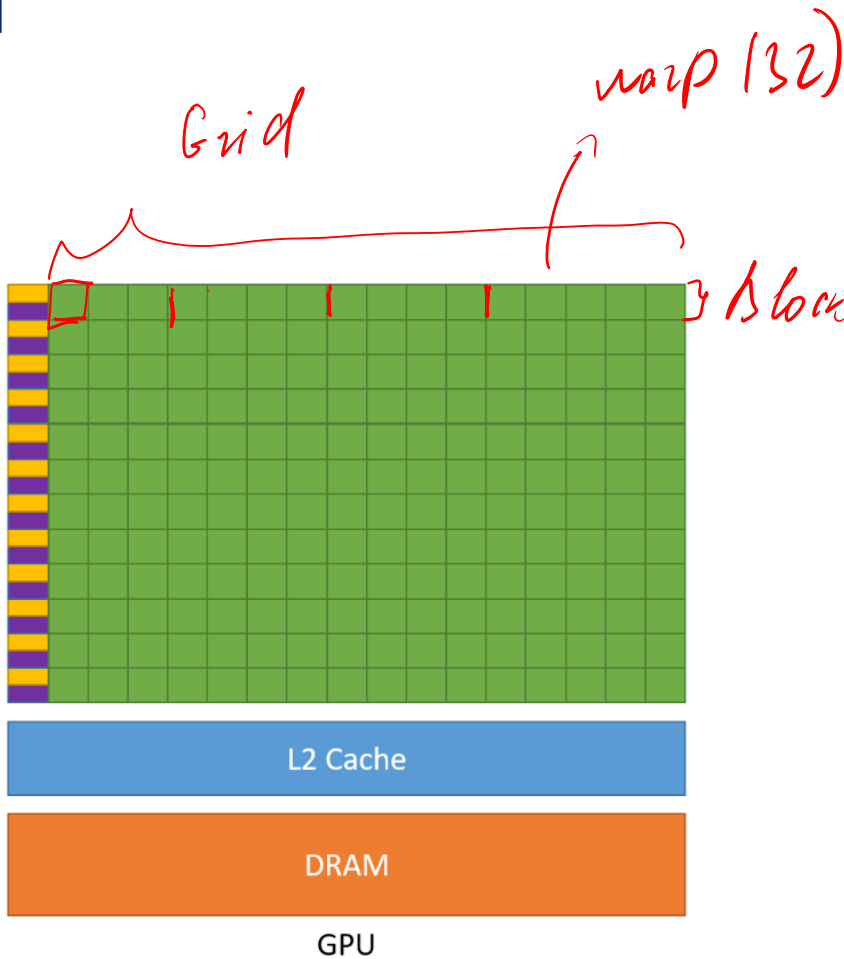
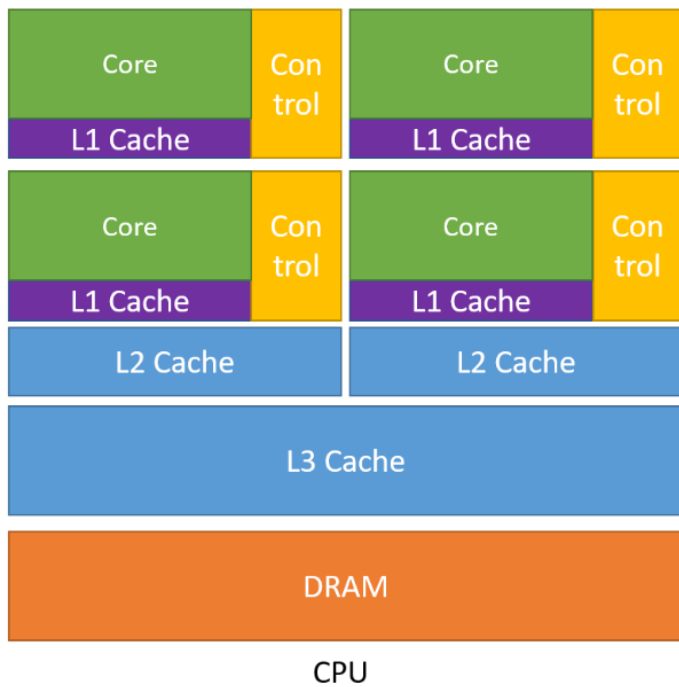


Figure 1: *The GPU Devotes More Transistors to Data Processing*

GPU Architecture Fundamentals

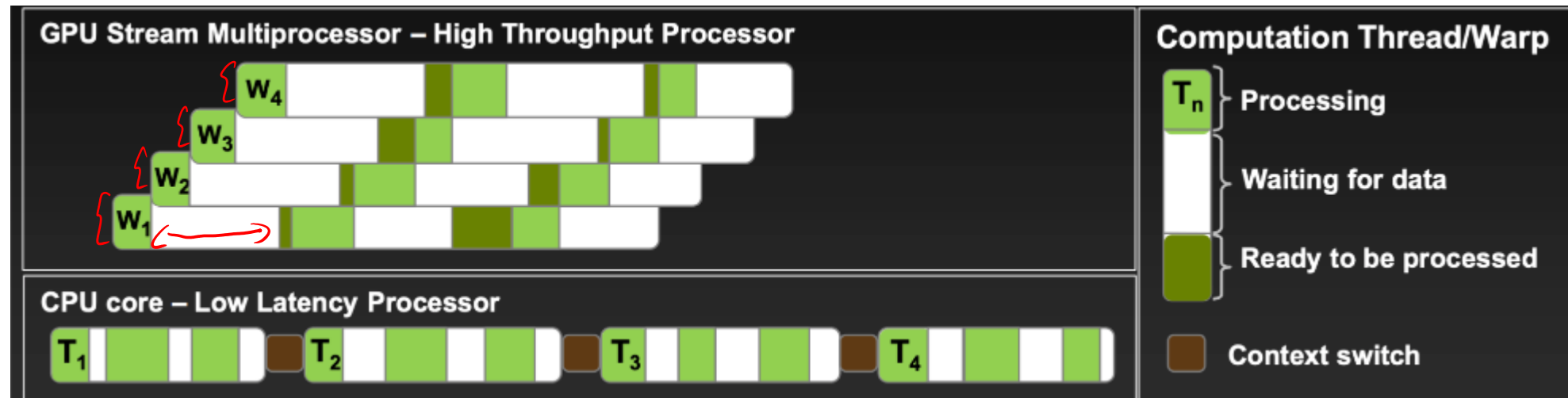


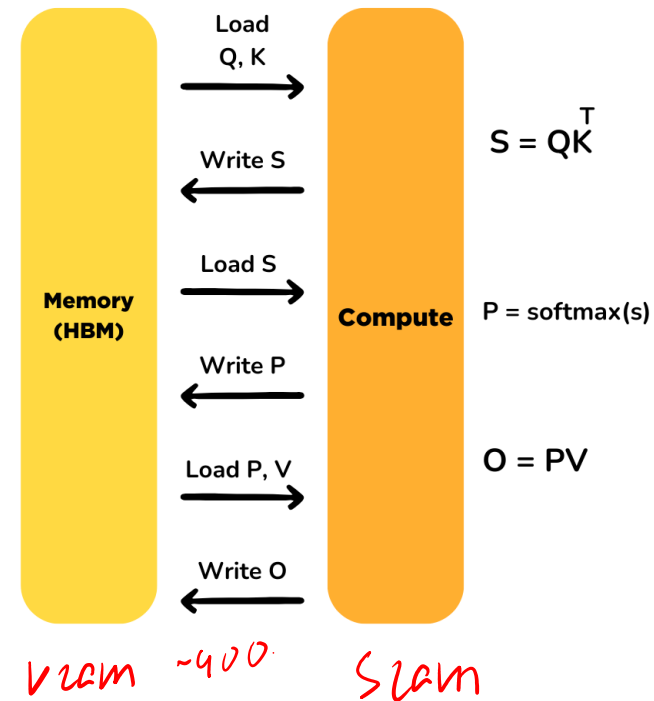


Table of Content

- GPU Architecture 101
- **Engineering tricks for attention**
- Intro to quantization
- Engineering tricks for decoding
- Miscellaneous

A vanilla attention hits a so-called Memory wall

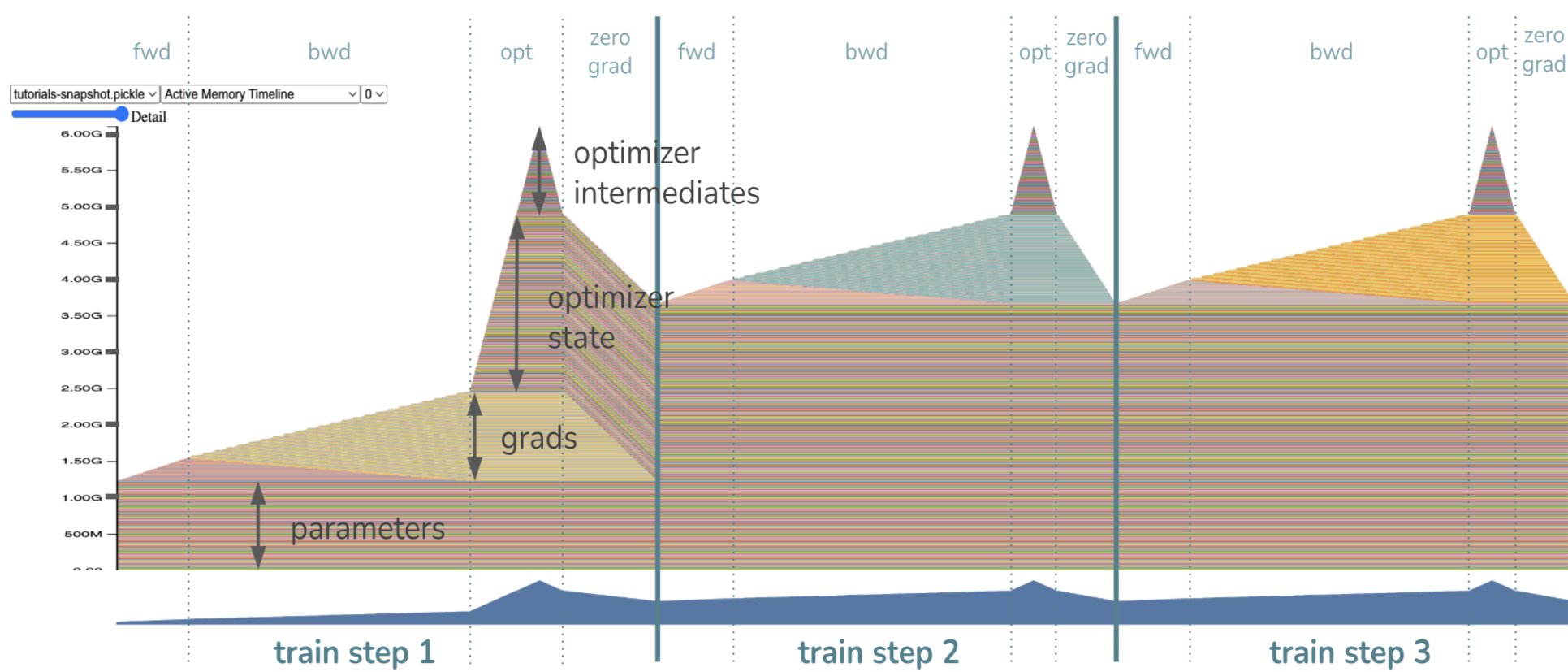
Standard Attention Implementation



$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Handwritten red notes: A squiggle under the softmax function and a 'P' below it.

Fuse optimizer's kernels into one

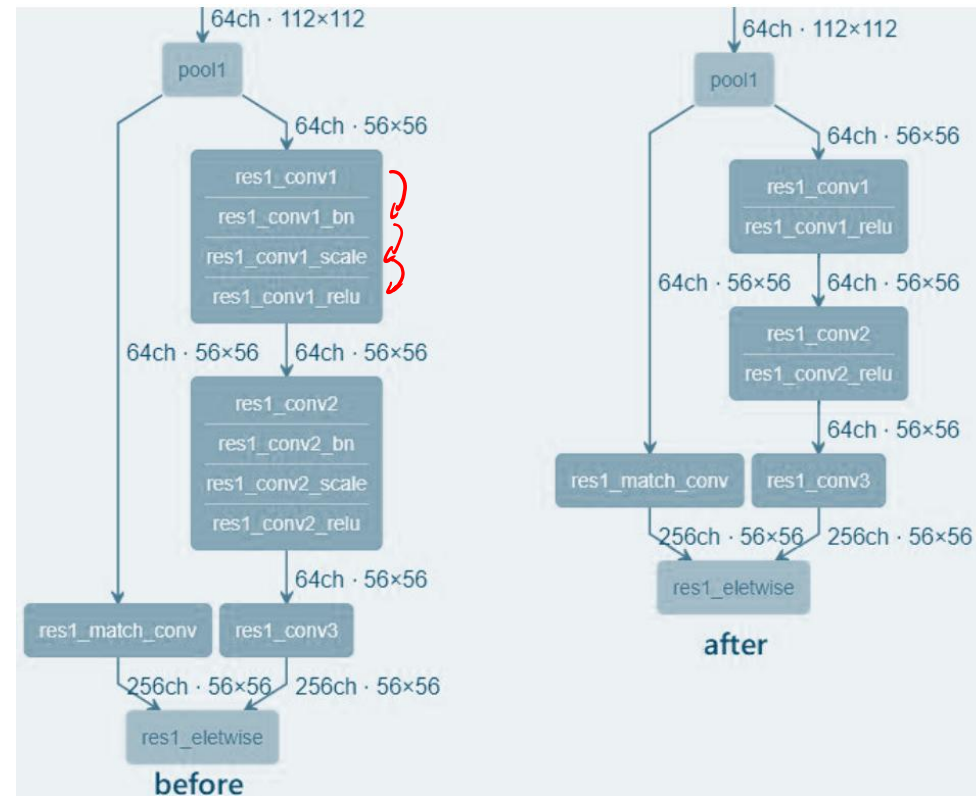


Fuse Torch modules

$A \cdot B = C$

$C \cdot D = E$

$A \cdot B \cdot D$

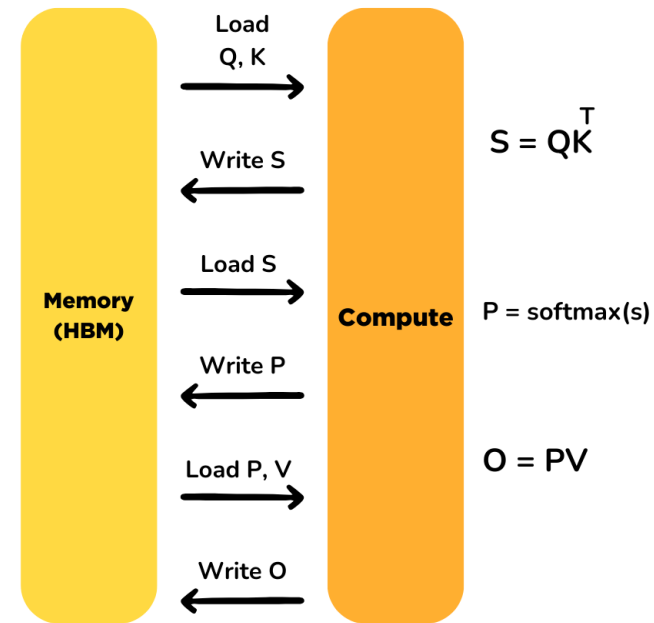


Key links:

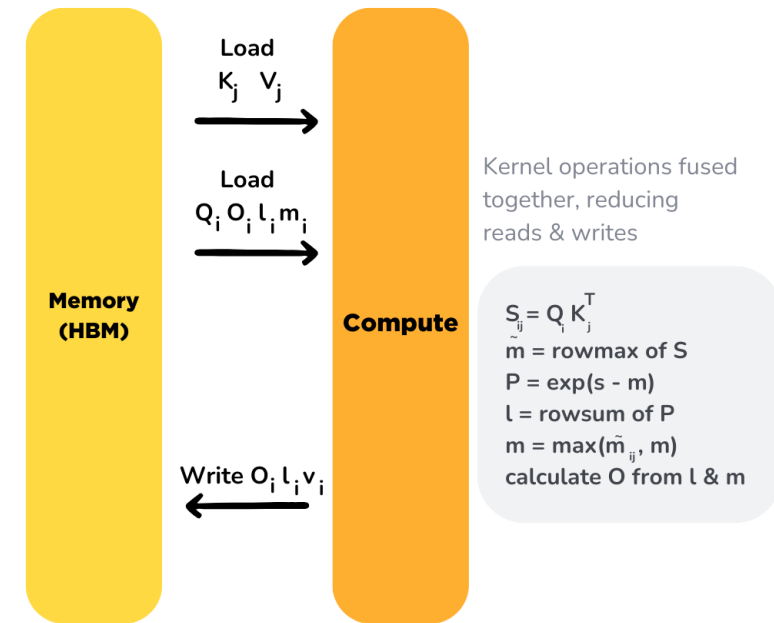
1: [Fuse Modules Recipe](#)

Flash Attention smartly bypasses heavy computations

Standard Attention Implementation

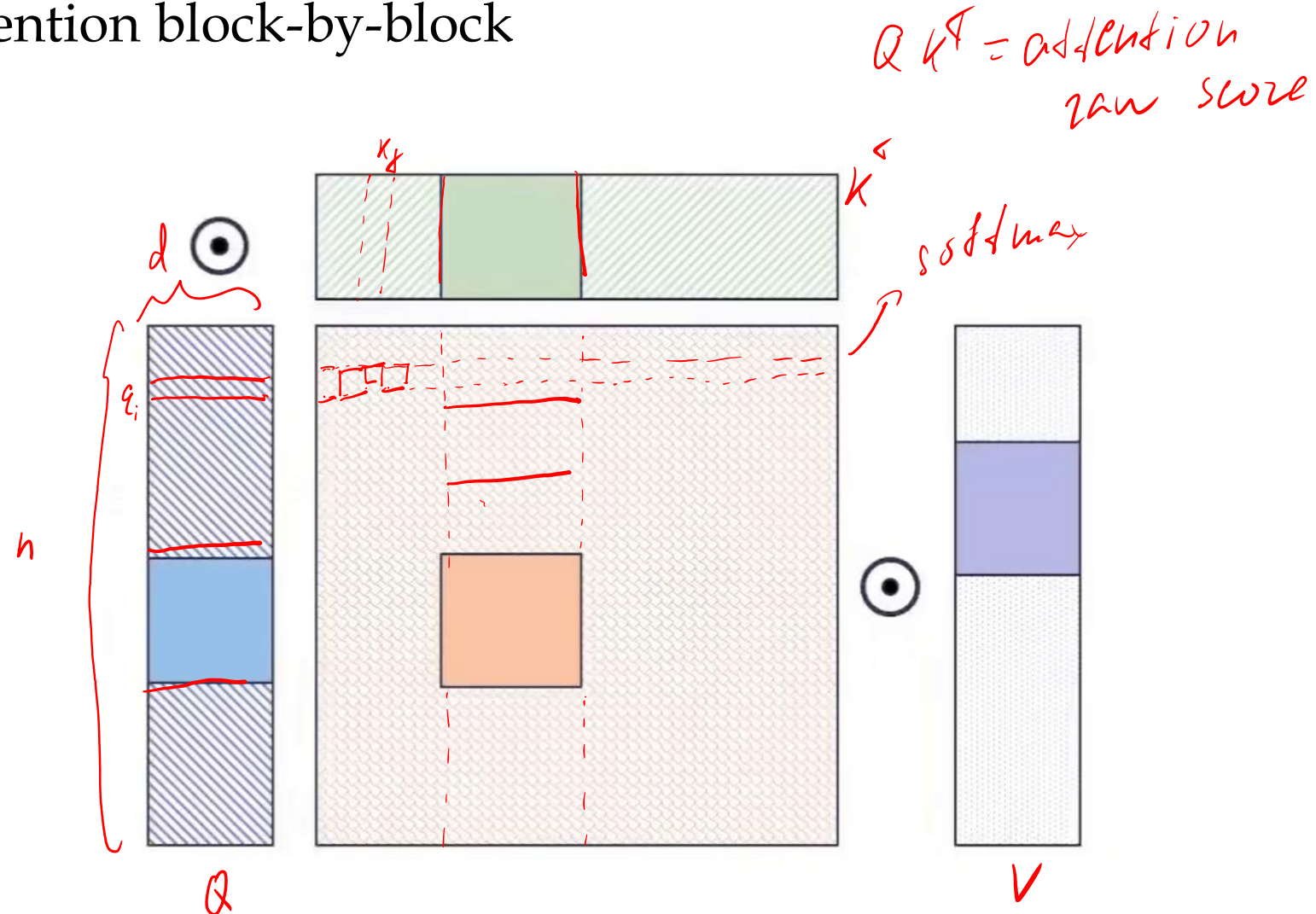


Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Calculate attention block-by-block



Transforming Softmax

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_k \exp(x_k)} \times \frac{\exp(-m)}{\exp(-m)}$$

Handwritten red notes: "QK" with an arrow pointing to the denominator, and "u" pointing to the index i.

$$e^{x_i} e^{-m} = e^{x_i - m}$$

$$\Rightarrow \frac{\exp(x_i - m)}{\sum_k \exp(x_k - m)}$$

Transforming Softmax

$$\begin{aligned}\text{Softmax}(x)_i &= \frac{\exp(x_i)}{\sum_k \exp(x_k)} \times \frac{\exp(-m)}{\exp(-m)} \\ &\Rightarrow \frac{\exp(x_i - m)}{\sum_k \exp(x_k - m)}\end{aligned}$$

Transforming Softmax

$$\text{Softmax}(x)_i = \frac{\exp(x_i)}{\sum_k \exp(x_k)} \times \frac{\exp(-m)}{\exp(-m)}$$

Handwritten red annotations: an arrow points from the second fraction to the word "max" written above it.

$$\Rightarrow \frac{\exp(x_i - m)}{\sum_k \exp(x_k - m)}$$

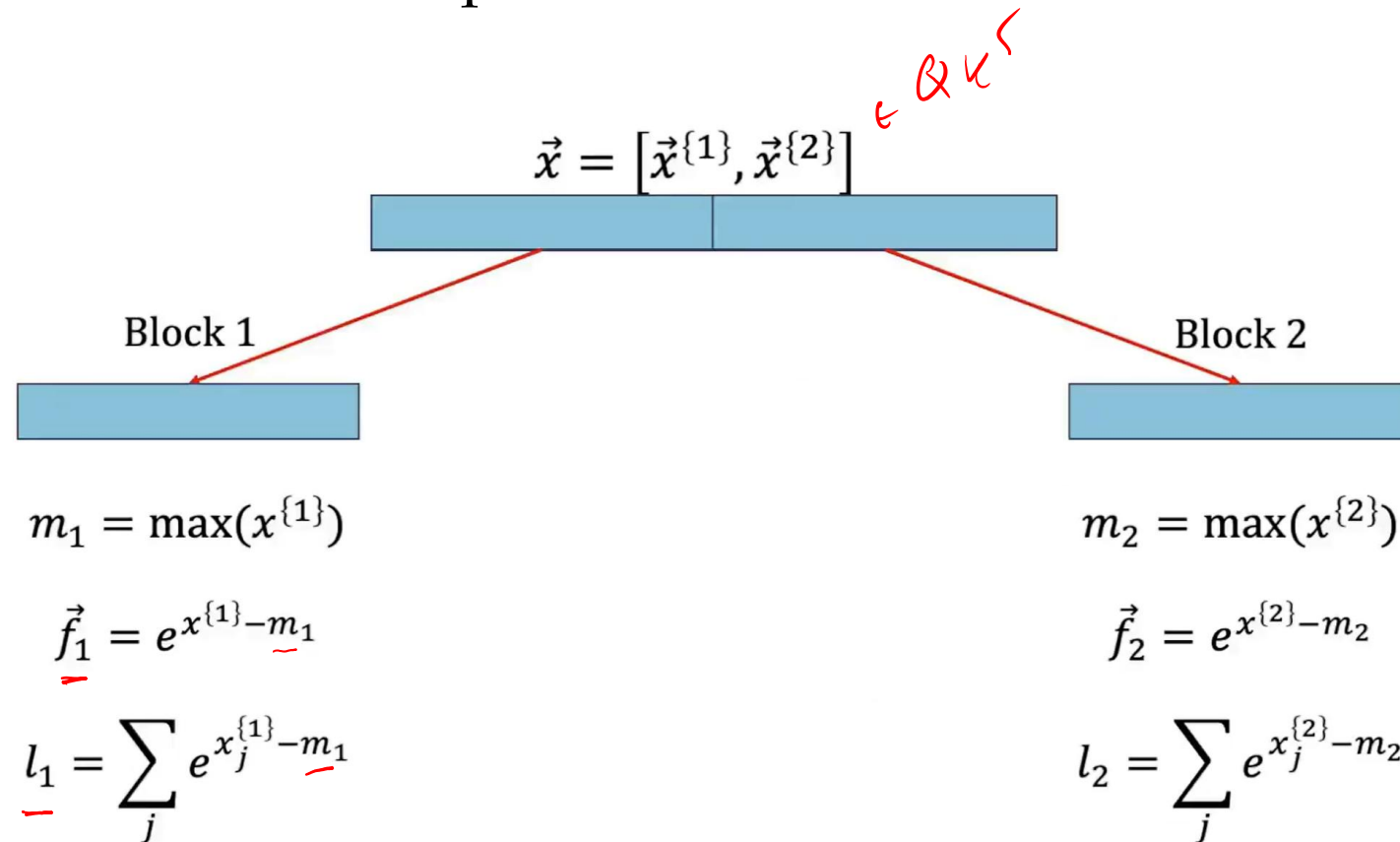
$$\vec{f}(x) = [e^{x_1-m}, e^{x_2-m}, e^{x_3-m}, \dots, e^{x_N-m}]$$

Handwritten red annotation: "QK" with an arrow pointing to the first term of the vector.

$$\underline{l(x)} = \sum_j f(x)_j$$

Handwritten red annotation: a wavy underline under the variable l(x).

Adapting to block-based computations



Softmax of two concatenated vectors

Block 1

$\vec{x}^{\{1\}}$

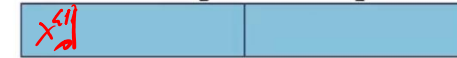
$$\left\{ \begin{array}{l} m_1 = \max(\vec{x}^{\{1\}}) \\ \underline{l_1} = \sum_j e^{x_j - m_1} \\ \vec{f_1} = e^{x^{\{1\}} - m_1} \end{array} \right.$$

Block 2

$\vec{x}^{\{2\}}$

$$\left\{ \begin{array}{l} m_2 = \max(\vec{x}^{\{2\}}) \\ \underline{l_2} = \sum_j e^{x_j - m_2} \\ \vec{f_2} = e^{x^{\{2\}} - m_2} \end{array} \right.$$

$$\vec{x} = [\vec{x}^{\{1\}}, \vec{x}^{\{2\}}]$$



Overall max

$$\underline{m(x)} = \max(m_1, m_2)$$

Adjusted $\vec{f_1}$ and $\vec{f_2}$

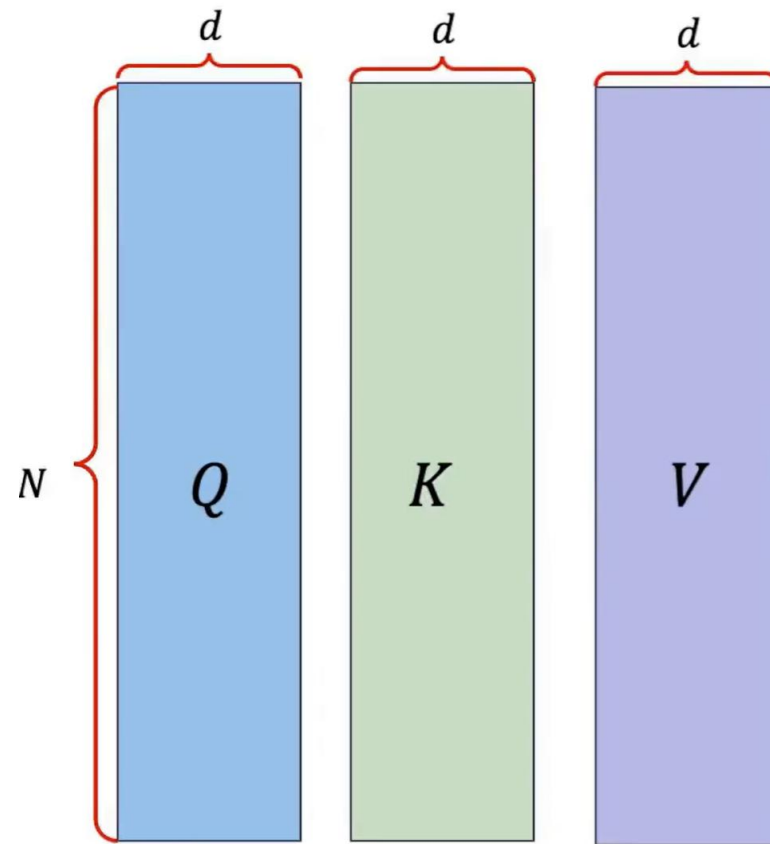
$$\underline{\vec{f}(x)} = [e^{m_1 - m(x)} \underline{f_1}, e^{m_2 - m(x)} \underline{f_2}]$$

Adjusted normalization factor

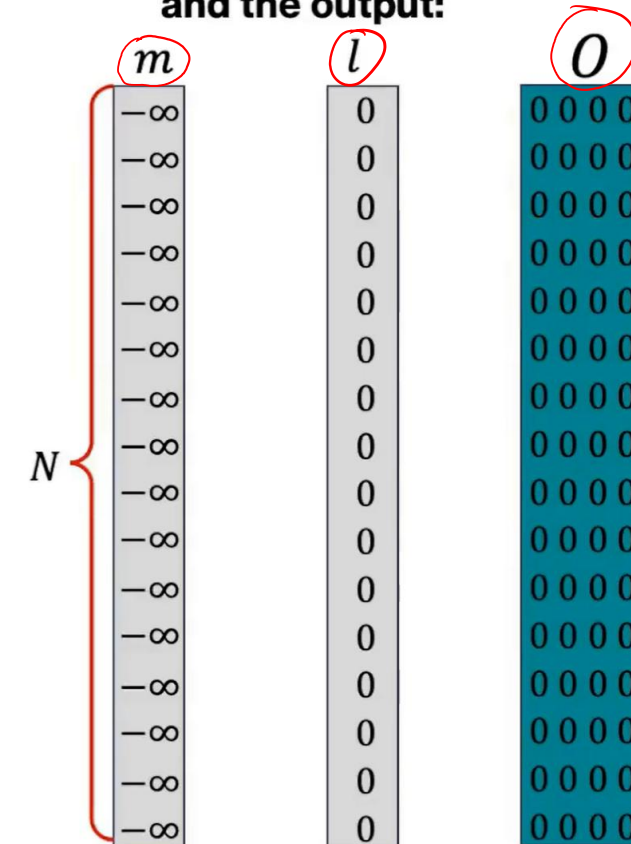
$$\underline{l(x)} = \underline{e^{m_1 - m(x)}} l_1 + e^{m_2 - m(x)} l_2$$

$$\vec{f}(x) = [e^{m_1 - m(x)} e^{x^{\{1\}} - m_1}, \dots]$$

Input and Output initialization

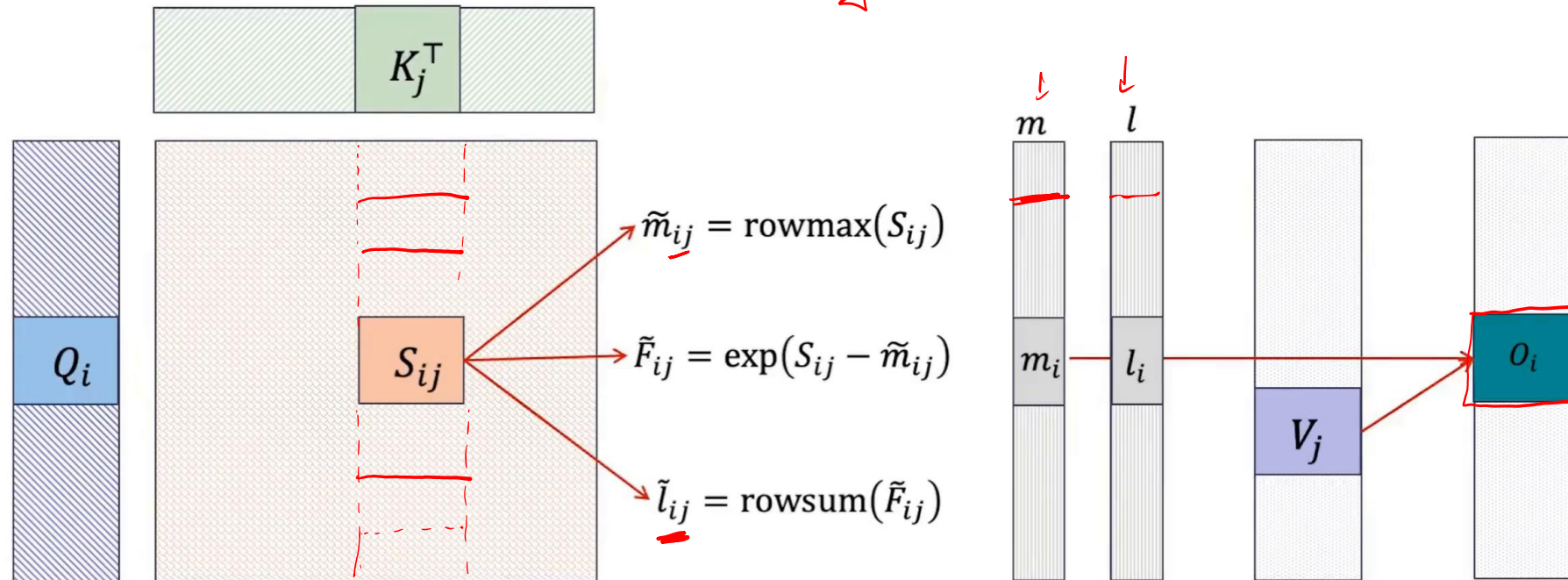


Initialize intermediate vectors
and the output:

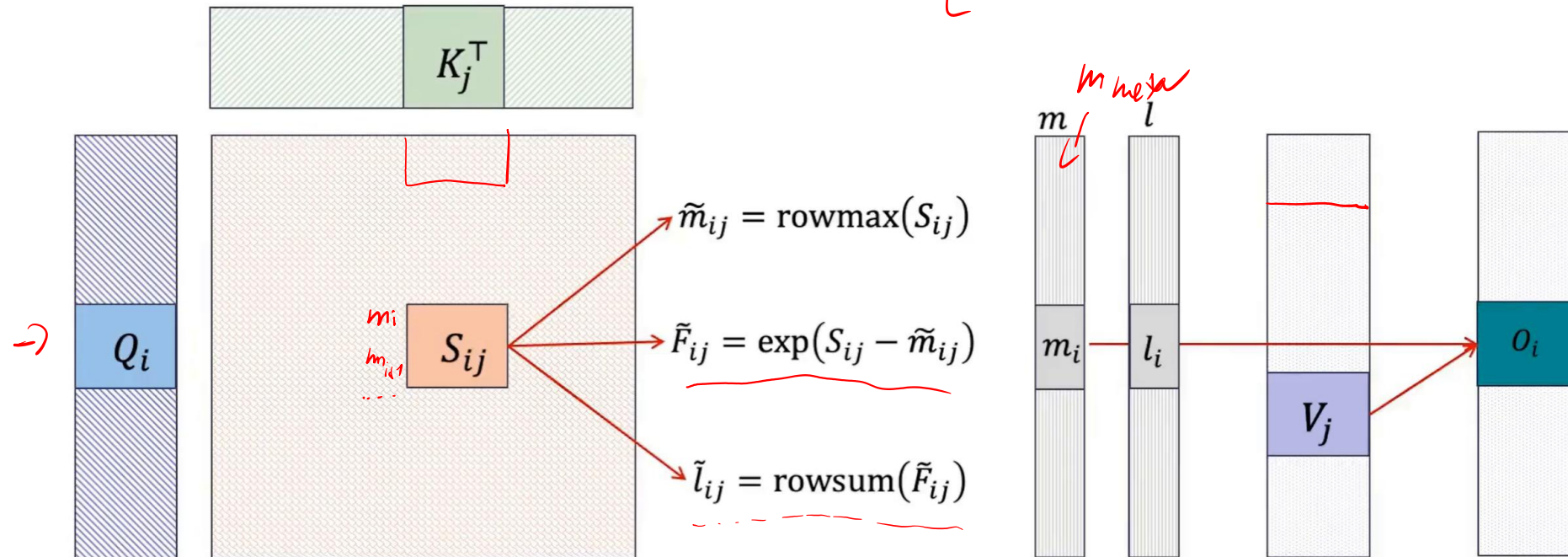


$\frac{1}{2} \log(2 \times 10^5) \checkmark$

Input and Output initialization

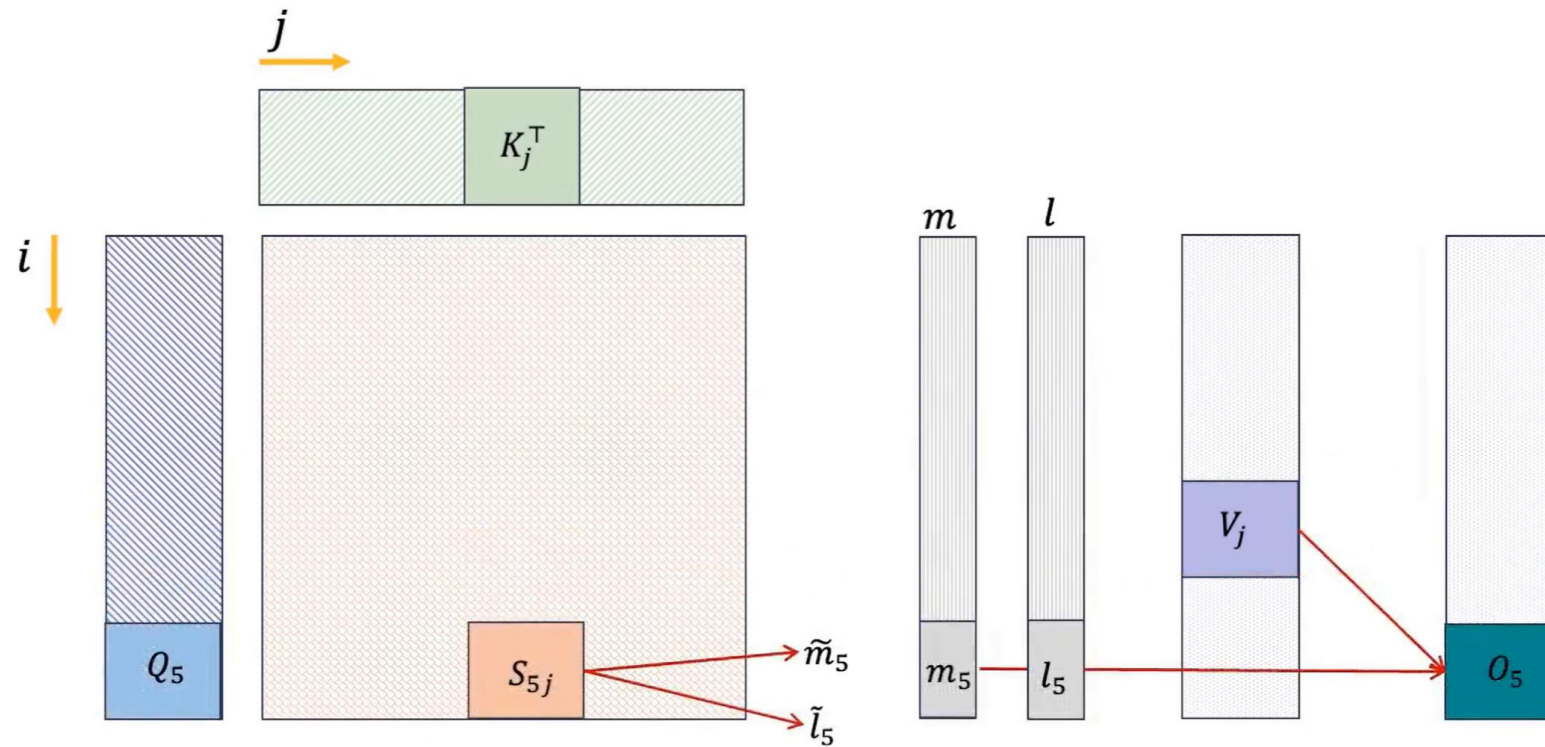


Input and Output initialization



$$O_i^{\text{new}} := \text{diag}(l_i^{\text{new}})^{-1} \left(\underbrace{\text{diag}(l_i)}_{\text{scale}} e^{m_i - m_i^{\text{new}}} O_i + \underbrace{e^{\tilde{m}_i - m_i^{\text{new}}} \tilde{F}_{ij} V_j}_{\text{update}} \right)$$

Partially update Outputs



Read/Write Complexity

Naïve Algorithm

$$O(Nd + N^2)$$

FlashAttention]

$$O(N^2 d^2 M^{-1})$$

M : Size of SRAM (LLM)
e.g. $M = 100\text{KB} \rightarrow M \ll d^2$

$$\frac{N^2 d^2}{M}$$

0. QK^T

1. softmax

3. $\cdot V$

3 ~~$N \times d$~~

Comparing to Vanilla

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	2.7 days (3.5×)
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	6.9 days (3.0×)

KV-cache

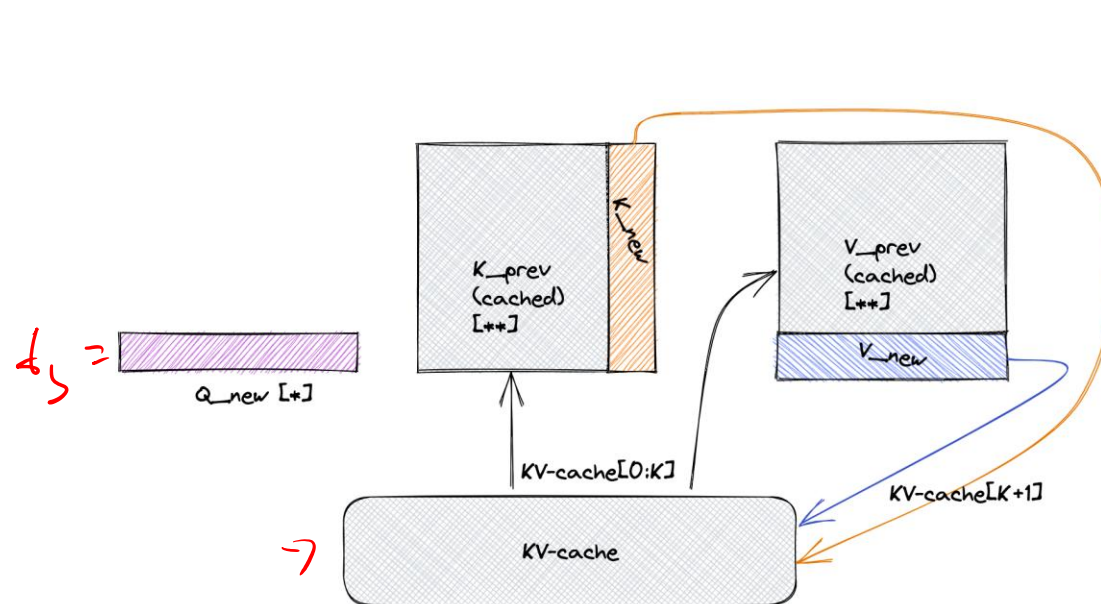
$$Q = \begin{bmatrix} & \\ & \end{bmatrix}$$

$$K = \begin{bmatrix} & \\ & \end{bmatrix}$$

$$V = \begin{bmatrix} & \\ & \end{bmatrix}$$

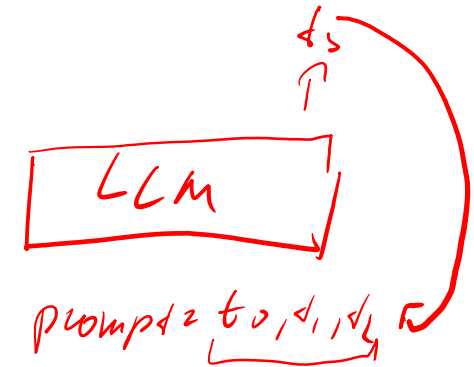
$$QK^T = n \begin{bmatrix} & \\ & \end{bmatrix} \xrightarrow{d} d \begin{bmatrix} & \\ & \end{bmatrix}$$

$d \cdot n \cdot n \gg O(n^2 d)$
 $\sim O(n^3)$



Notes:

- * When processing token[K], we only need the K'th row of Q
- ** When processing token[K], we require the full K & V tensors, but we can mostly reuse the cached values (This enables skipping the computation of K & V)



Key links:

- 1: [Key-Value Cache Quantization](#)

Paged attention

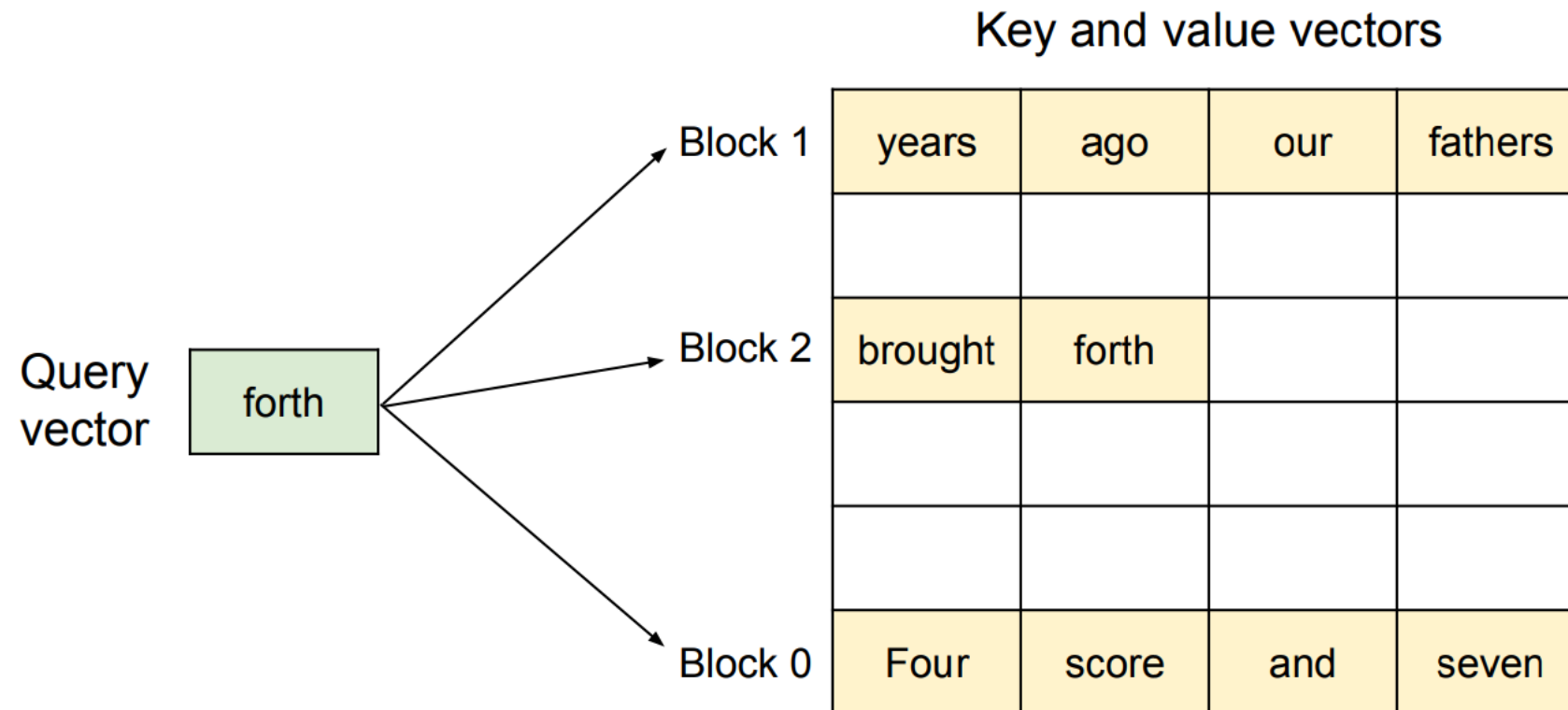




Table of Content

- GPU Architecture 101
- Engineering tricks for attention
- **Intro to quantization**
- Engineering tricks for decoding
- Miscellaneous

GPU Architecture Fundamentals

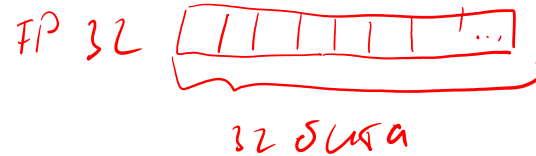


Figure 2. Multiply-add operations per clock per SM

	CUDA Cores				Tensor Cores					
NVIDIA Architecture	FP64	FP32	FP16	INT8	FP64	TF32	FP16	INT8	INT4	INT1
Volta	32	64	128	256			512			
Turing	2	64	128	256			512	1024	2048	8192
Ampere (A100)	32	64	256	256	64	512	1024	2048	4096	16384
Ampere, sparse						1024	2048	4096	8192	

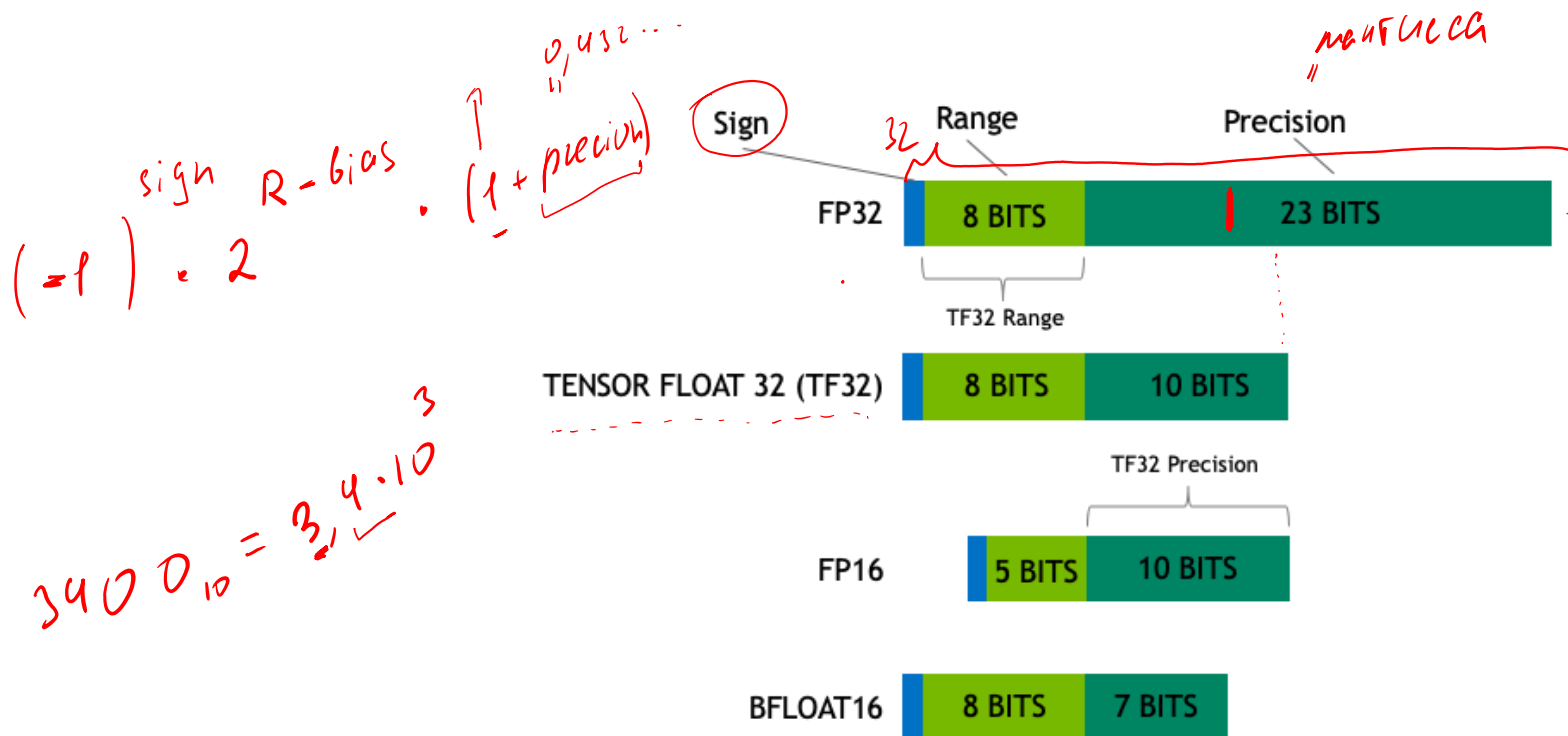
cores
64 • # SM • # clocks per second

GPU Architecture Fundamentals

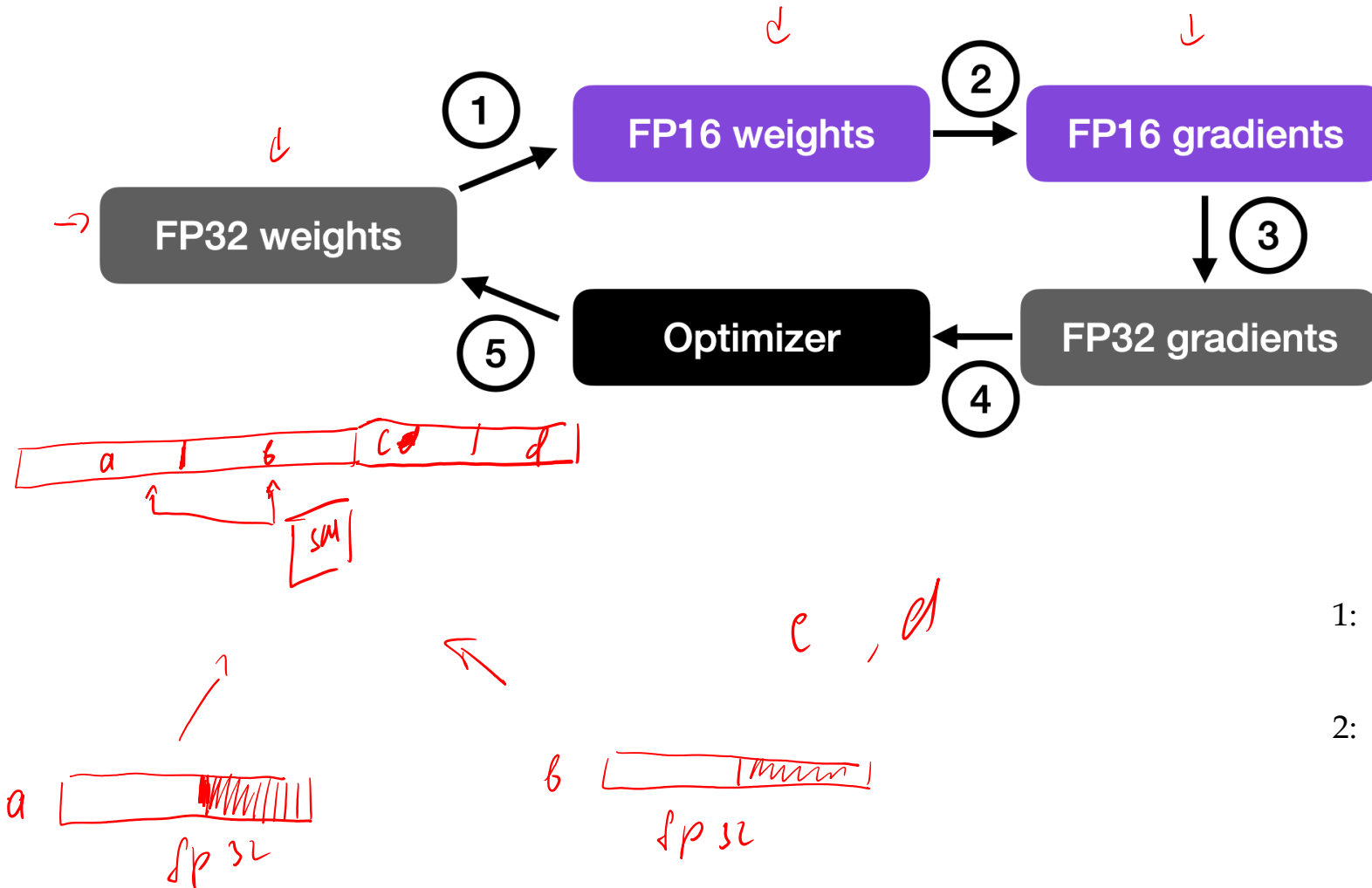
	H100 SXM	H100 NVL
FP64	34 teraFLOPS	30 teraFLOPs
FP64 Tensor Core	67 teraFLOPS	60 teraFLOPs
FP32	67 teraFLOPS	60 teraFLOPs
TF32 Tensor Core*	989 teraFLOPS	835 teraFLOPs
BFLOAT16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS	3,341 teraFLOPS
INT8 Tensor Core*	3,958 TOPS	3,341 TOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	3.35TB/s	3.9TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG

Image source: <https://www.nvidia.com/en-us/data-center/h100/>

Machine numbers refresher



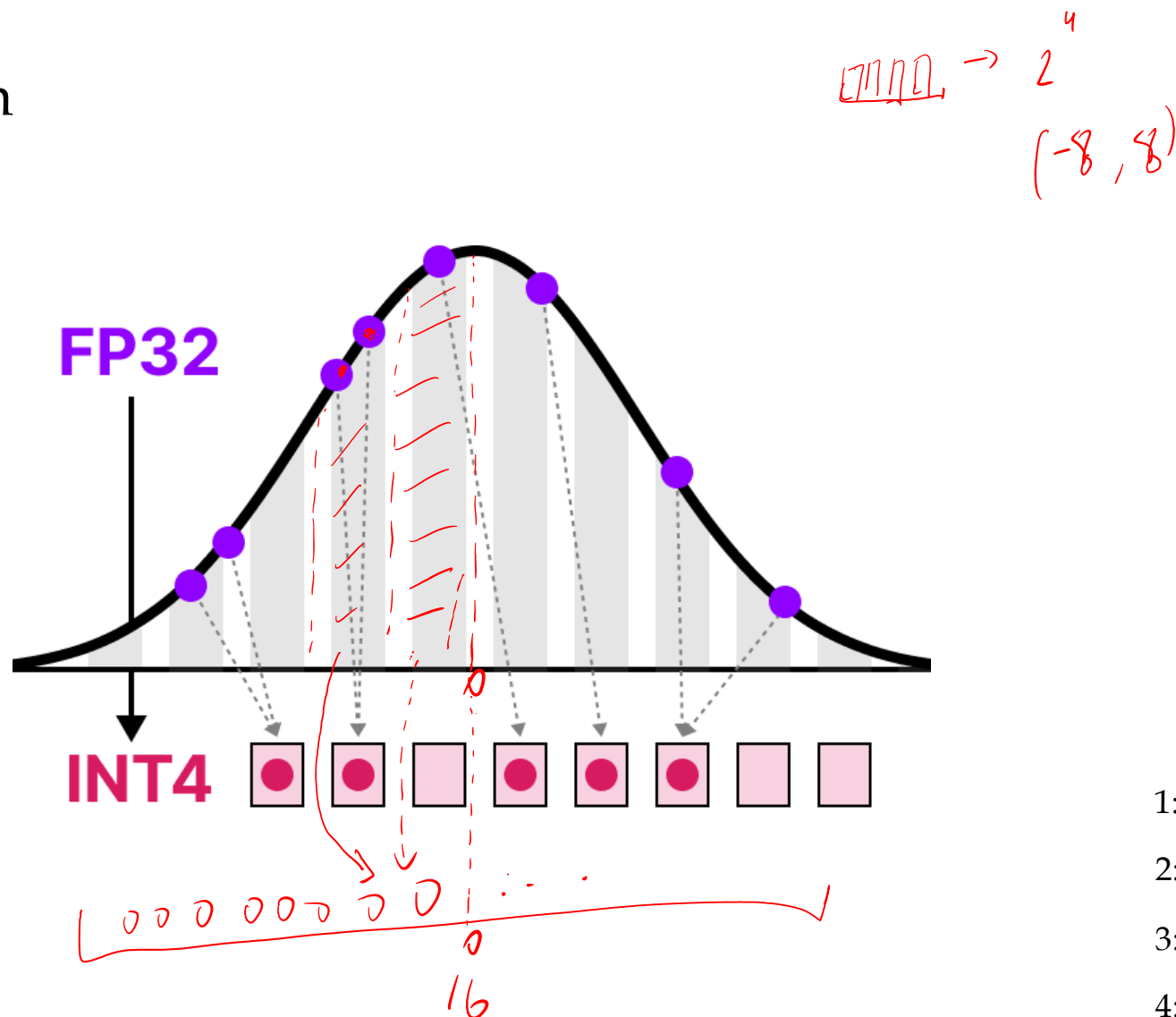
Mixed Precision



Key links:

- 1: [Automatic Mixed Precision in Torch](#)
- 2: [What Every User Should Know About Mixed Precision Training in PyTorch](#)

Quantization



Key links:

- 1: [Quantization Recipe](#)
- 2: [HF Optimum](#)
- 3: [Practical Quantization](#)
- 4: [HF bitsandbytes integration](#)

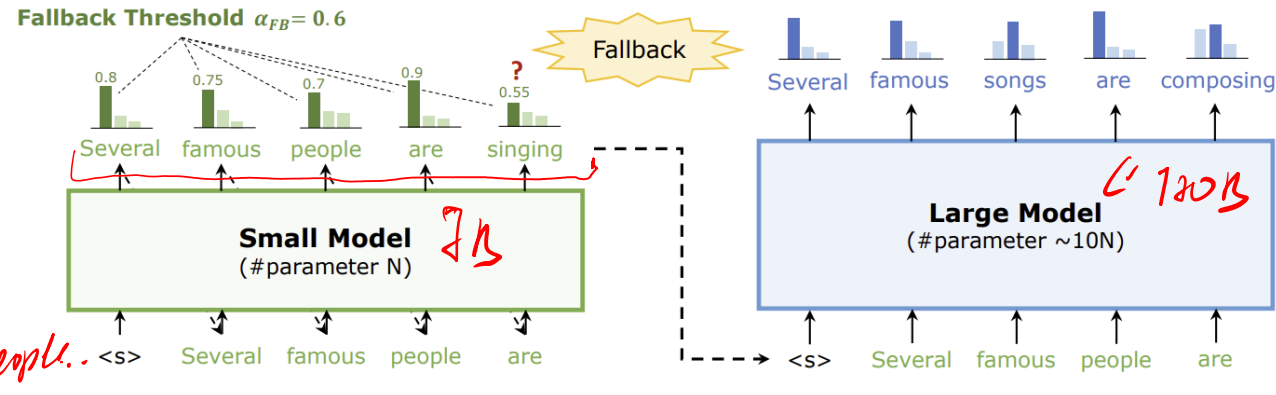


Table of Content

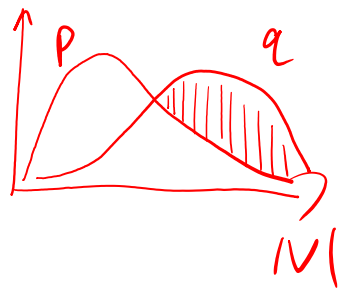
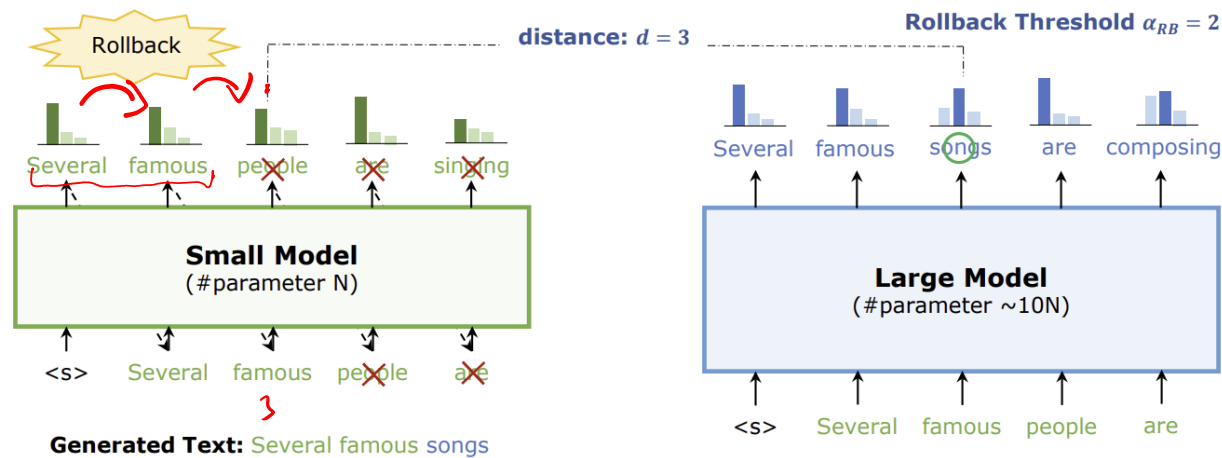
- GPU Architecture 101
- Engineering tricks for attention
- Intro to quantization
- **Engineering tricks for decoding**
- Miscellaneous

Speculative Decoding

Several
Several famous
Several famous people.



river bank is flooded
bank of china



$$P_3 > Q_3; Q_3 > P_3$$

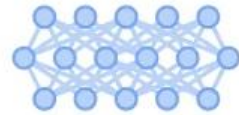
$\frac{P_3}{Q_3}$

Key links:

- 1: [Speculative Decoding paper](#)
- 2: [A Hitchhiker's Guide to Speculative Decoding](#)

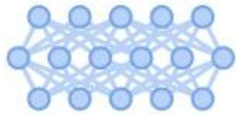
Speculative Decoding

WITHOUT SPECULATIVE DECODING



My favorite thing about fall is

WITH SPECULATIVE DECODING



My favorite thing about fall is the change in the leaves. The trees

Speculative Decoding

Table 2. Empirical results for speeding up inference from a T5-XXL 11B model.

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

Adaptive Batching

Batch 0. & больше и мур*...
1. ~~короткий, loss, loss~~
2. новый batch

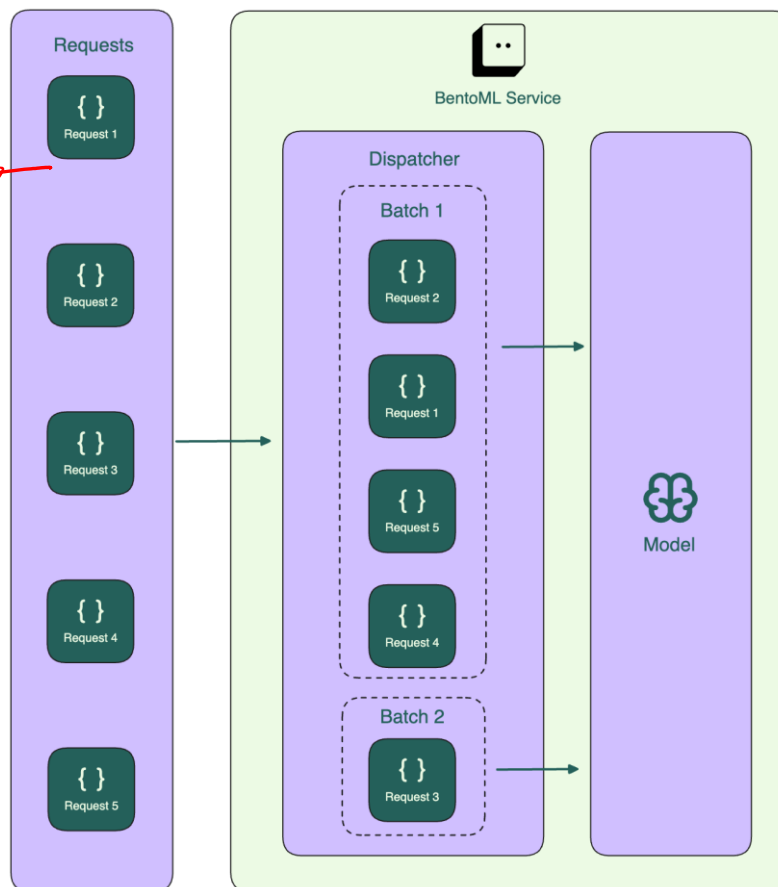
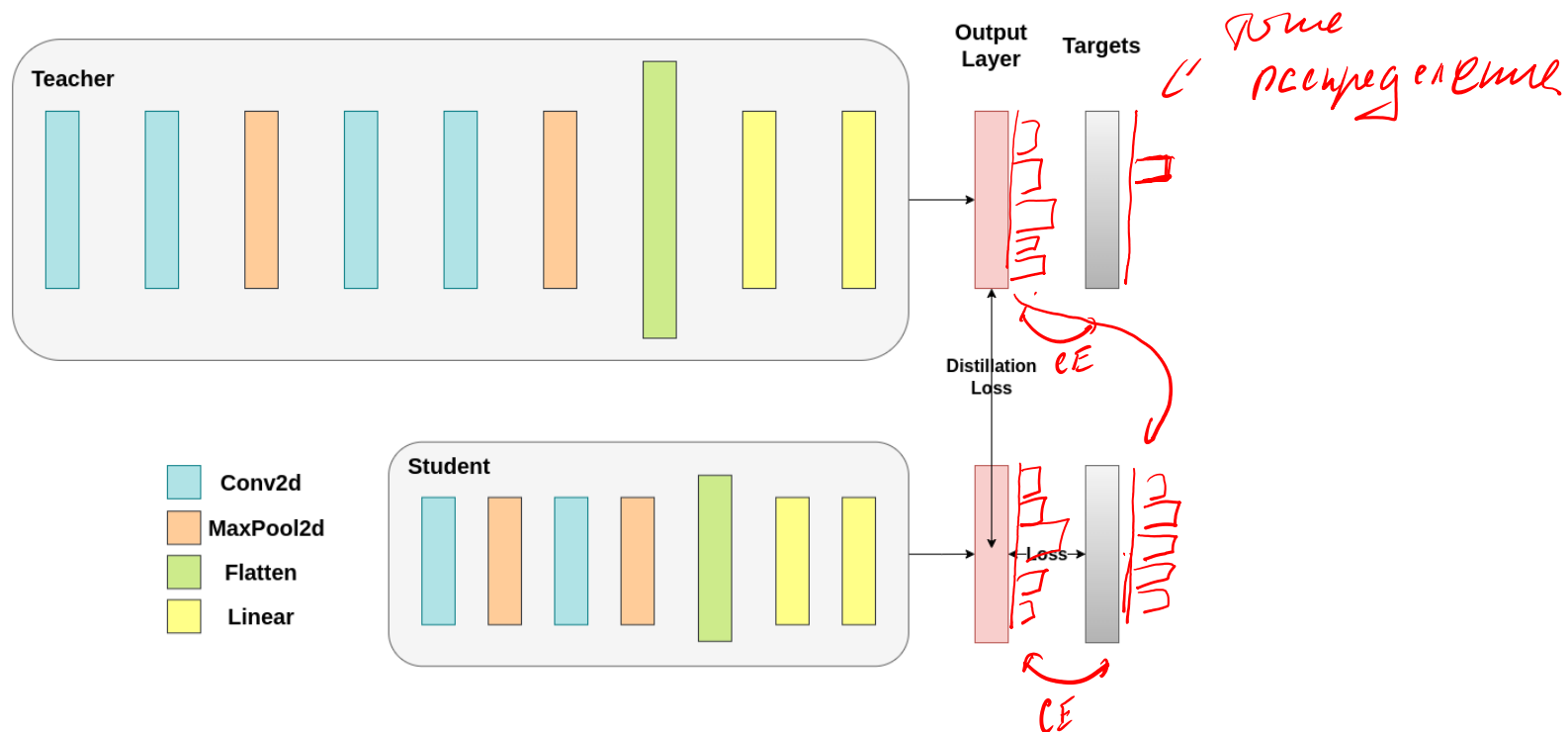




Table of Content

- GPU Architecture 101
- Engineering tricks for attention
- Intro to quantization
- Engineering tricks for decoding
- **Miscellaneous**

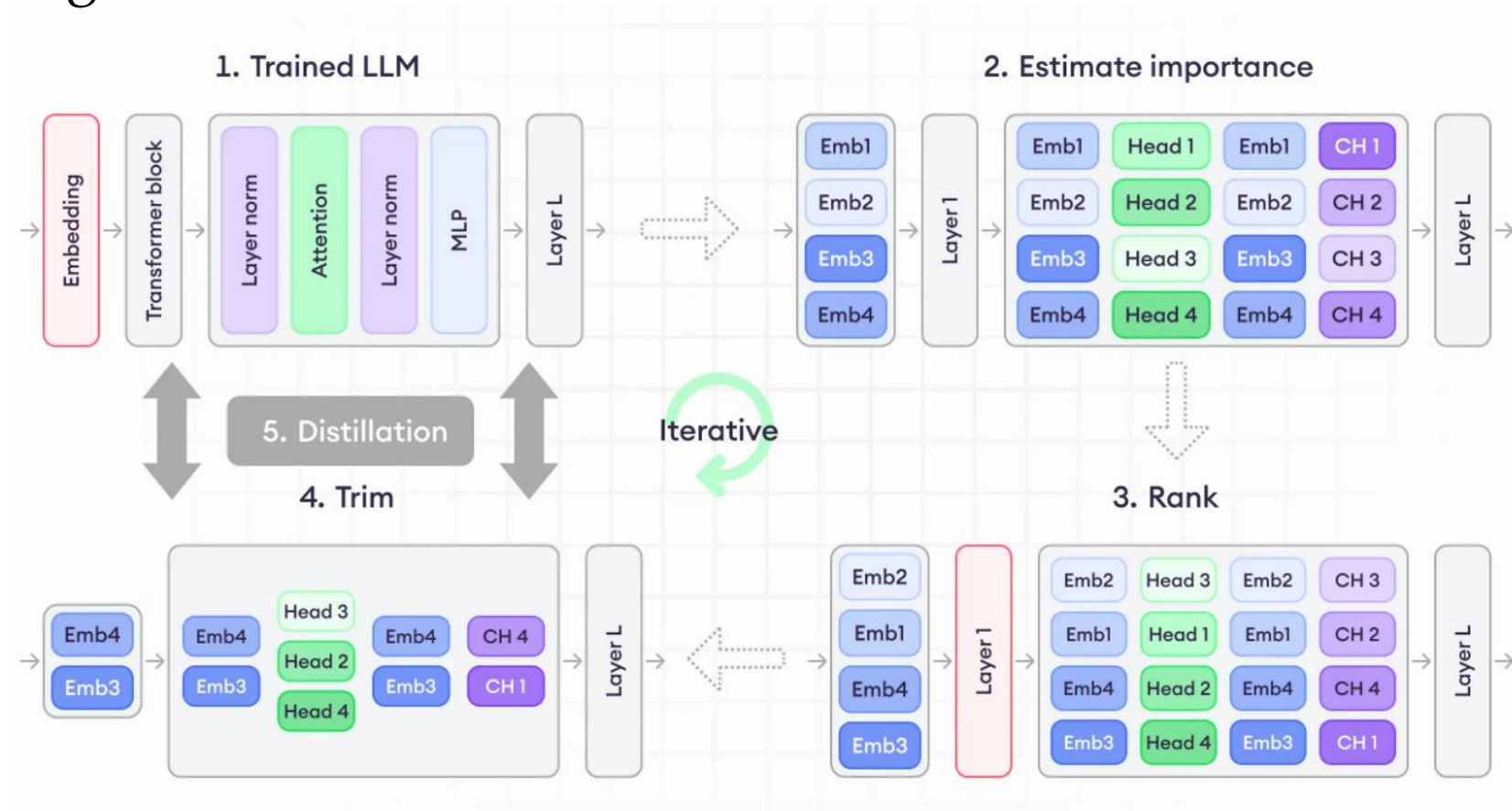
Knowledge Distillation



Key links:

1: [Distillation Tutorial Torch](#)

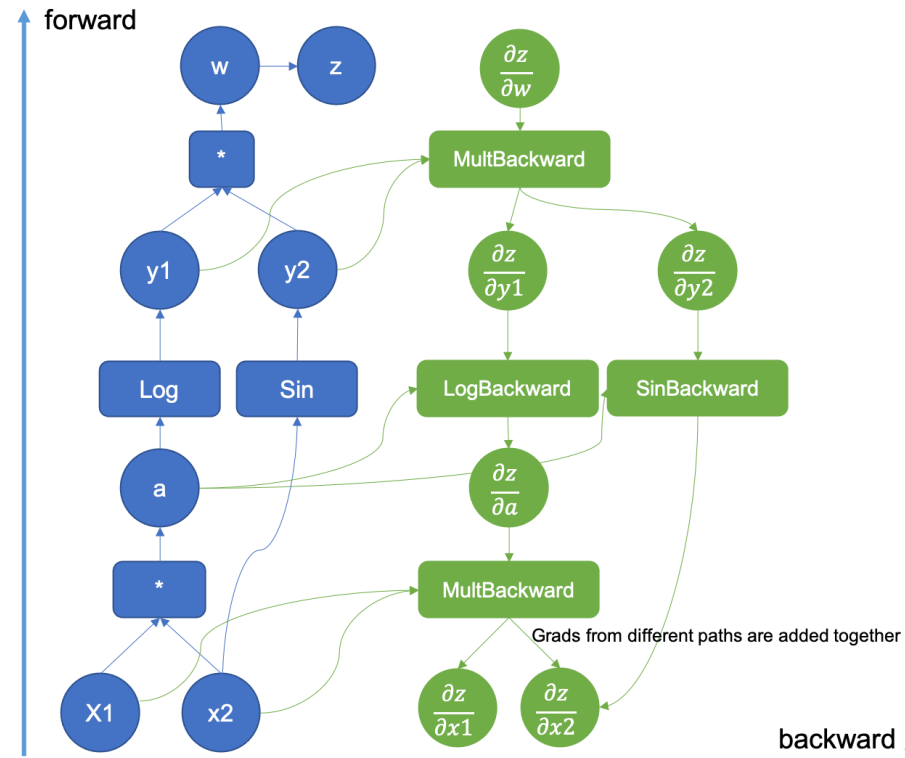
Pruning



Key links:

1: [Pruning Tutorial Torch](#)

Make Torch's Dynamic Computation Graph Static



Key links:

1: [torch.compile](https://pytorch.org/docs/stable/torch_compile.html)

